# Sets, relations, functions, graphs, and trees

*Part VIII Appendix*

## Scope

Use elementwise logic and general proofs to connect sets, equivalence partitions, functions, graphs, and trees. State graph, path, direction, empty-set, and binary-slot conventions explicitly. Prove the six equivalent tree conditions and the bipartite criterion.

## Prerequisites

Basic discrete-mathematics notation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Appendix route

*Learning objectives*

- Apply definitions of sets, relations, and functions precisely.
- Recognize graph and tree structure.
- Connect Cartesian products, equivalence relations, and recursive structures.
- B.1 Sets
- B.2 Relations
- B.3 Functions
- B.4 Graphs
- B.5 Trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Sets

A set is unordered and has no repetition, so $\{u, v, u\} = \{v, u\}$. A multiset retains multiplicity, while a sequence retains positions. These are different objects. Membership is written $x \in A$. The condition $A \subseteq B$ means that every $x \in A$ also lies in $B$; $A \subsetneq B$ additionally requires $A \neq B$.

The empty set is a subset of every set $A$, but need not be an element of $A$. The set $\{\varnothing\}$ has one element and is not empty. Equality requires both inclusions, $A \subseteq B$ and $B \subseteq A$.

Here $\mathbb{N} = \{0, 1, \ldots\}$, $\mathbb{Z}$ denotes the integers, and $\mathbb{R}$ denotes the reals. The expression $\{x \in \mathbb{Z} : 3 \mid x\}$ specifies a subset by a property, rather than an ordered list.

Set operations express elementwise logic. Membership in $A \cup B$ uses "or," membership in $A \cap B$ uses "and," and $x \in A \setminus B$ means $x \in A$ and $x \notin B$. A complement requires a specified universe $U$, with $A^c = U \setminus A$. Changing $U$ changes the complement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Set identities quantify over every element

| Property | Union form | Intersection form |
|---|---|---|
| Identity and zero | $A \cup \varnothing = A$ | $A \cap \varnothing = \varnothing$ |
| Idempotence | $A \cup A = A$ | $A \cap A = A$ |
| Commutativity | $A \cup B = B \cup A$ | $A \cap B = B \cap A$ |
| Associativity | $A \cup (B \cup C) = (A \cup B) \cup C$ | $A \cap (B \cap C) = (A \cap B) \cap C$ |
| Distributivity | $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ | $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ |
| Absorption | $A \cup (A \cap B) = A$ | $A \cap (A \cup B) = A$ |
| De Morgan | $(B \cup C)^c = B^c \cap C^c$ | $(B \cap C)^c = B^c \cup C^c$ |

For an arbitrary element $x$,

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Set identities quantify over every element (continued)

$$
\begin{aligned}
x \in A \setminus (B \cap C) &\iff x \in A \ \wedge \ \neg(x \in B \wedge x \in C) \\
&\iff (x \in A \wedge x \notin B) \ \vee \ (x \in A \wedge x \notin C) \\
&\iff x \in (A \setminus B) \cup (A \setminus C).
\end{aligned}
$$

This two-way equivalence proves the identity for every $x$. A finite Venn example cannot replace that universal quantifier.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Partitions, cardinality, and overlapping counts

A partition of $S$ is a collection of nonempty, pairwise disjoint blocks whose union is $S$. Every $x \in S$ belongs to exactly one block. For $S = \varnothing$, the empty collection of blocks is allowed. Congruence modulo $3$ partitions $\{0, \dots, 7\}$ into $\{0, 3, 6\}$, $\{1, 4, 7\}$, and $\{2, 5\}$.

For finite $A, B$, split their union into the disjoint parts $A \setminus B$, $A \cap B$, and $B \setminus A$. This gives

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

For $A = \{0, 2, 4, 6\}$ and $B = \{2, 3, 6\}$, the union has size $4 + 3 - 2 = 5$. Simply adding cardinalities counts the intersection twice.

Three-set inclusion-exclusion adds the three singleton sizes, subtracts the three pair intersections, and adds the triple intersection. In general, an element that appears in $r \geq 1$ sets receives coefficient

$$\sum_{j=1}^{r} (-1)^{j+1} \binom{r}{j} = 1,$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

because $(1-1)^r = 0$. An element in no set receives coefficient $0$. This proves inclusion-exclusion element by element.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Power sets versus Cartesian products

The power set is $\mathcal{P}(S) = \{T : T \subseteq S\}$. For $n$ distinguishable elements, each subset chooses whether each element is included. This is a bijection with the $n$-bit strings, so $|\mathcal{P}(S)| = 2^n$. In particular, $\mathcal{P}(\varnothing) = \{\varnothing\}$ has one element.

An ordered pair $(a, b)$ retains positions and generally differs from $(b, a)$. Define

$$A \times B = \{(a, b) : a \in A,\ b \in B\}.$$

For finite sets, each fixed $a$ gives $|B|$ pairs. The $|A|$ disjoint rows give $|A \times B| = |A||B|$. An $n$-fold product counts ordered tuples and multiplies the finite factor sizes. The set $A^0$ contains the unique empty tuple, so it has size $1$.

For $A = \{p, q\}$ and $B = \{0, 1, 2\}$, $A \times B$ has $6$ elements. By contrast, $\mathcal{P}(A)$ has the four elements $\varnothing$, $\{p\}$, $\{q\}$, and $\{p, q\}$. A sequence is a function from positions to values. Equal values at distinct positions remain distinct entries.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Infinite cardinality requires bijections

Two sets have equal cardinality when a bijection exists. Being infinite alone does not establish equal cardinality. Define a map from $\mathbb{N}$ to $\mathbb{Z}$ by

$$f(0) = 0, \qquad f(2k-1) = k, \qquad f(2k) = -k \quad (k \geq 1).$$

Every integer has exactly one preimage, so $\mathbb{Z}$ is countable. The map $n \mapsto 2n$ is a bijection onto the even natural numbers. An infinite set can have the same cardinality as a proper subset.

To prove that $\mathbb{R}$ is uncountable, suppose $(0, 1)$ has a list $x_1, x_2, \ldots$. Choose decimal representations that do not end in infinitely many $9$s. Define the $i$th digit of $y$ to be $1$ unless the $i$th digit of $x_i$ is $1$, in which case choose $2$. Then $y \in (0, 1)$, its representation is unambiguous, and it differs from every $x_i$ at digit $i$. This contradicts the claimed complete list.

For finite sets of equal size, injection implies surjection. For infinite sets, $f(n) = n + 1$ on $\mathbb{N}$ is injective but misses $0$. The finite premise is essential.

# Relations

A binary relation is a subset $R \subseteq A \times B$. A relation on one set has $R \subseteq A \times A$. Write $aRb$ when $(a, b) \in R$. A relation need not assign a partner to every input, and a partner need not be unique.

| Property | Requirement for all $a, b, c \in A$ |
|---|---|
| Reflexive | $aRa$ |
| Symmetric | $aRb \implies bRa$ |
| Antisymmetric | $(aRb \land bRa) \implies a = b$ |
| Transitive | $(aRb \land bRc) \implies aRc$ |

Symmetry and antisymmetry are not negations. Equality satisfies both. For distinct $a, b, c$, a relation containing $a \leftrightarrow b$ and $b \to c$, but lacking $c \to b$, can satisfy neither. On a nonempty $A$, the empty relation is vacuously symmetric, antisymmetric, and transitive, but is not reflexive. Symmetry and transitivity do not guarantee the existence of any related pair.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Equivalence relations induce partitions

An equivalence relation is reflexive, symmetric, and transitive. Define $[a] = \{b \in A : aRb\}$. Reflexivity makes each class nonempty and ensures that every $a$ is covered.

Suppose $c \in [a] \cap [b]$. Then $aRc$ and $bRc$ imply $aRb$ by symmetry and transitivity. For any $x \in [a]$, symmetry gives $bRa$, and transitivity with $aRx$ gives $bRx$. Thus $[a] \subseteq [b]$. Exchanging $a$ and $b$ gives the reverse inclusion. Classes coincide or are disjoint, and removing duplicates yields a partition.

Conversely, given a partition, define $aRb$ exactly when $a$ and $b$ share a block. Every element lies in one block, proving reflexivity. Sharing a block is symmetric. If $a, b$ share one block and $b, c$ share another, uniqueness of the block containing $b$ makes all three share a block. This proves transitivity. The block containing $a$ is exactly $[a]$, so the constructions correspond.

Congruence modulo $3$ illustrates the axioms. The difference $a - b$ is a multiple of $3$. Zero proves reflexivity, negation proves symmetry, and addition proves transitivity. The integer classes have residues $0, 1, 2$; restricting them to the earlier finite set gives its three blocks.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Partial orders, total orders, and extrema

A partial order is reflexive, antisymmetric, and transitive. A total order also compares every pair. A total preorder requires reflexivity, transitivity, and comparability, but permits distinct elements tied in both directions.

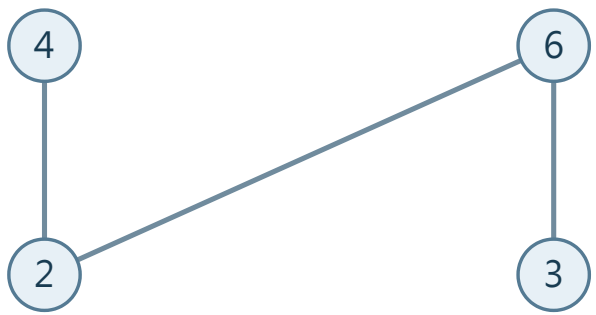
A maximum $M$ satisfies $x \leq M$ for every $x$. A maximal element $m$ merely has no strictly larger element. Minimum and minimal are the dual notions. If $M, N$ are both maxima, then $M \leq N$ and $N \leq M$, so antisymmetry gives $M = N$. A maximum is maximal, but a maximal element need not be a maximum.

On $S = \{2, 3, 4, 6\}$ ordered by divisibility, the minimal elements are $2, 3$ and the maximal elements are $4, 6$. Neither a minimum nor a maximum exists. A Hasse edge is a cover $x < y$ with no $z$ satisfying $x < z < y$. Draw larger elements above smaller ones and omit reflexive and transitive edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Partial orders, total orders, and extrema (continued)

S={2,3,4,6}: divisibility Hasse diagram




Upward: larger divisible element

covers：2–4、2–6、3–6

minimal：2、3

maximal：4、6

No minimum or maximum

Lines are covers, not numerical distances. Reflexive and transitive edges are omitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Functions

A function $f : A \to B$ is a relation assigning exactly one $b \in B$ to every $a \in A$. The set $A$ is the domain, $B$ is the codomain, and the range $f(A)$ may be smaller than $B$. A partial function undefined at some $a$ is not a total function under this definition.

An injection satisfies $f(a) = f(a') \implies a = a'$. A surjection gives every $b \in B$ at least one preimage. A bijection satisfies both properties. The map $f(n) = 2n$ from $\mathbb{N}$ to $\mathbb{N}$ is injective but not onto. Changing its codomain to the nonnegative even integers makes it bijective. The map $n \mapsto \lfloor n/2 \rfloor$ from $\mathbb{N}$ to $\mathbb{N}$ is onto but not injective, since both $0$ and $1$ map to $0$.

For finite sets, an injection uses distinct codomain elements, so $|A| \leq |B|$. A surjection must supply every codomain element, so $|A| \geq |B|$. With equal finite sizes, either property implies the other. The infinite counterexample above prevents extending that inference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Inverses, preimages, and composition

For a bijection, $f^{-1}(b)$ is the unique $a$ with $f(a) = b$, and

$$f^{-1} \circ f = \mathrm{id}_A, \qquad f \circ f^{-1} = \mathrm{id}_B .$$

An injection alone has an inverse only on its range. Without injection, the inverse relation generally is not a function.

For any function and $T \subseteq B$, the set preimage is

$$f^{-1}(T) = \{a \in A : f(a) \in T\}.$$

It is defined without invertibility. It shares notation with an inverse function but is a different object.

For $f : A \to B$ and $g : B \to C$, composition satisfies $(g \circ f)(a) = g(f(a))$. If both maps are injective, equality of composite values lets us cancel $g$, then $f$, proving equality of inputs. If both are onto, given $c \in C$, choose $b \in B$ with $g(b) = c$, then $a \in A$ with $f(a) = b$. This proves surjectivity of the composite. For bijections,

$$(g \circ f)^{-1} = f^{-1} \circ g^{-1},$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Inverses, preimages, and composition (continued)

with the order reversed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Graph conventions and degree sums

A finite digraph $G = (V, E)$ has $E \subseteq V \times V$, permitting self-loops but no parallel edges. A simple undirected graph uses unordered pairs of distinct vertices and has no loops. A multigraph must preserve edge identities and specify how loops contribute to degree.

A directed edge $(u, v)$ leaves $u$ and enters $v$. A self-loop contributes one to each of indegree and outdegree. Undirected degree counts incident edges. Each undirected edge has two endpoints, while each directed edge has one tail and one head. Thus

$$\sum_{v \in V} \deg(v) = 2|E|,$$

$$\sum_{v \in V} \operatorname{outdeg}(v) = \sum_{v \in V} \operatorname{indeg}(v) = |E|.$$

The first identity implies that the number of odd-degree vertices is even. Moving vertices or bending edges does not change $E$. Crossing lines are not junctions unless marked as vertices. Legends or edge lists must determine directions, identities, and values.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Walks, simple paths, and cycles

A walk $v_0, \ldots, v_k$ follows an edge between each adjacent pair. Its length is $k$ edges, rather than $k+1$ vertices. CLRS permits repetition in a path; that object is called a walk here. A simple path has distinct vertices. Every vertex has a length-zero path to itself.

For undirected acyclicity, a cycle has length at least $3$ and repeats only the start and end vertex. Traversing one edge from $u$ to $v$ and back is not a cycle. Directed simple cycles may have length $1$ for a loop or $2$ for opposite arcs, and must follow every direction. A general closed walk may repeat other vertices and differs from a simple cycle.

If a walk from $u$ to $v$ repeats $v_i = v_j$ with $i < j$, remove the closed segment between those occurrences. The remaining sequence is a shorter walk with the same endpoints. Repeated removal strictly decreases a finite length and eventually leaves a simple path. This proves equivalence of reachability by walks and by simple paths without listing infinitely many walks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Components, SCCs, and direction

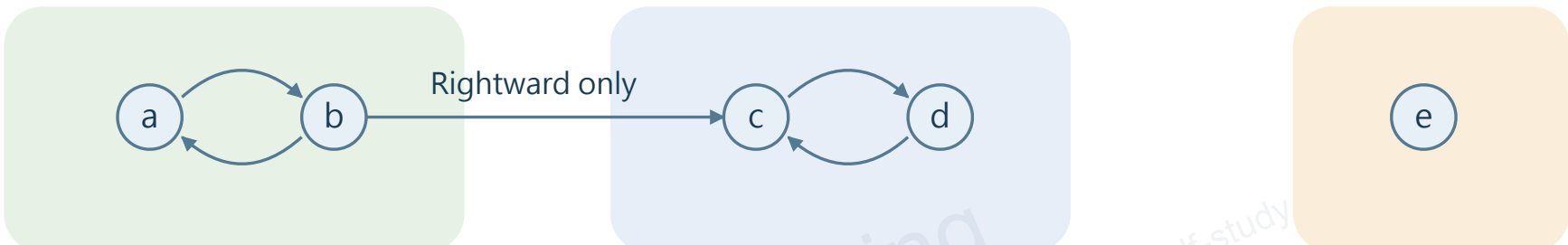
Undirected reachability is reflexive by zero-length paths, symmetric by reversal, and transitive by concatenation. Components are its equivalence classes. Directed reachability is reflexive and transitive, but need not be symmetric. Mutual directed reachability has all three properties; its classes are strongly connected components, or SCCs.

Take the arcs $a \to b$, $b \to a$, $b \to c$, $c \to d$, $d \to c$, and an isolated vertex $e$. The SCCs are $\{a, b\}$, $\{c, d\}$, and $\{e\}$. Vertex $a$ reaches $d$, but $d$ cannot reach $a$, so the first two SCCs remain distinct. Ignoring directions gives only the components $\{a, b, c, d\}$ and $\{e\}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Components, SCCs, and direction (continued)

Reachability is directional; mutual reachability differs




a reaches d; d cannot reach a. Ignore direction: a/b/c/d together, e still isolated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Subgraphs, isomorphism, and contraction

A subgraph selects vertices and some original edges, keeping both endpoints of every selected edge. An induced subgraph $G[W]$ keeps all original edges with both endpoints in $W$. For the preceding example, $W = \{a, b, c\}$ induces exactly $a \to b$, $b \to a$, and $b \to c$.

An isomorphism is a vertex bijection $f : V \to V'$ satisfying

$$(u, v) \in E \iff (f(u), f(v)) \in E'.$$

Preservation in one direction alone is not the general definition. Equal vertex counts, edge counts, and degree multisets are necessary but insufficient. The cycle $C_6$ and two disjoint triangles both have $6$ vertices, $6$ edges, and degree $2$ everywhere, but only the cycle is connected.

Contracting an undirected edge $\{u, v\}$ merges its endpoints into $x$, redirects other incident edges, and removes the contracted edge. A simple representation merges parallel copies to a common neighbor and discards loops. Algorithms that need parallel edges, including some randomized contractions, must explicitly use a multigraph instead of silently deduplicating.

## Graph classes and the bipartite criterion

The complete graph $K_n$ joins every distinct pair and has $n(n-1)/2$ edges. A bipartite graph has a partition $V = L \sqcup R$ with every edge crossing the two sides, equivalently a proper two-coloring. A DAG is directed and acyclic; trees and forests here are undirected. Absence of directed cycles alone does not make a tree.

A bipartite graph has no odd cycle. Every edge switches sides, and an odd number of switches cannot return to the original side. Conversely, suppose there is no odd simple cycle. Root each component and color vertices by the parity of their shortest undirected distance from the root. An edge joining equal parities would create an odd closed walk by following a root-to-$u$ path, the edge, and a reversed root-to-$v$ path.

If that odd closed walk repeats an internal vertex, split there into two shorter closed walks. Their lengths sum to an odd number, so one is odd. Repeatedly keep the odd part until no internal repetition remains. This is an odd simple cycle, a contradiction. Hence every edge crosses colors, proving bipartiteness.

# Graph classes and the bipartite criterion (continued)

A hyperedge may contain many vertices. Its incidence bipartite graph puts original vertices on one side and hyperedges on the other, with an edge for each membership. These are different node types; a hyperedge is not an ordinary vertex pair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Trees and forests

A free tree here is a nonempty finite simple undirected graph that is connected and has no simple cycle. A forest is acyclic and may contain multiple components or isolated vertices. Empty forests are allowed. Empty graphs are outside the six-equivalence theorem below.

Every tree with $n \geq 2$ has a leaf. Take an endpoint $v$ of a longest simple path. A neighbor outside the path would extend it. An additional neighbor elsewhere on the path would create a cycle. Thus $v$ has only its adjacent path neighbor and has degree $1$.

Removing a leaf and its edge preserves connectivity because simple paths between other vertices cannot use a leaf internally. It also preserves acyclicity. Starting with one vertex and zero edges, induction gives $n - 1$ edges in a tree. For a forest with $c$ components, sum the component tree counts:

$$|E| = \sum_{i=1}^{c} (n_i - 1) = n - c.$$

The identity includes the empty forest, where $n = c = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Six equivalent tree conditions

For a nonempty finite simple undirected graph with $n$ vertices and $m$ edges, these conditions are equivalent:

1. The graph is a tree.
2. Each pair of vertices has exactly one simple path between them.
3. The graph is connected, and deleting any edge disconnects it.
4. The graph is connected, with $m = n - 1$.
5. The graph is acyclic, with $m = n - 1$.
6. The graph is acyclic, and adding any missing edge creates a cycle.

$(1) \implies (2)$**.** Connectivity gives a path. If two distinct simple paths existed, their first divergence and subsequent first meeting would delimit internally disjoint segments forming a cycle. This contradicts acyclicity.

$(2) \implies (3)$**.** An edge $\{u, v\}$ is itself the unique simple $u$-to-$v$ path. Deleting it destroys that reachability.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Six equivalent tree conditions (continued)

$(3) \implies (1)$**.** If a cycle existed, deleting one of its edges would leave the other arc as a replacement in any path using that edge. The graph would remain connected, a contradiction. The leaf-induction count gives $(1) \implies (4)$ and $(1) \implies (5)$. Existence of a path and uniqueness of that path are separate assertions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Remaining directions and a spanning-tree construction

Every connected graph has a spanning tree. Start with one visited vertex, forming a set $S$. While $S \neq V$, connectivity supplies an edge from $S$ to its complement. Add that edge and its one new endpoint. Each step attaches one new vertex and creates no cycle. After $n - 1$ steps, the selected edges form a spanning tree with $n - 1$ edges. In particular, every connected graph has $m \geq n - 1$.

$(4) \implies (1)$**.** The constructed spanning tree uses all $n - 1$ graph edges. There are no extra edges, so the graph equals that tree.

$(5) \implies (1)$**.** The forest identity $m = n - c$, together with $m = n - 1$, gives $c = 1$, proving connectivity.

$(1) \implies (6)$**.** Every new edge $\{u, v\}$ closes the tree's unique $u$-to-$v$ path into a cycle.

$(6) \implies (1)$**.** If two components existed, adding an edge between them could not create a cycle. This contradicts condition $(6)$, so the graph is connected.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Remaining directions and a spanning-tree construction (continued)

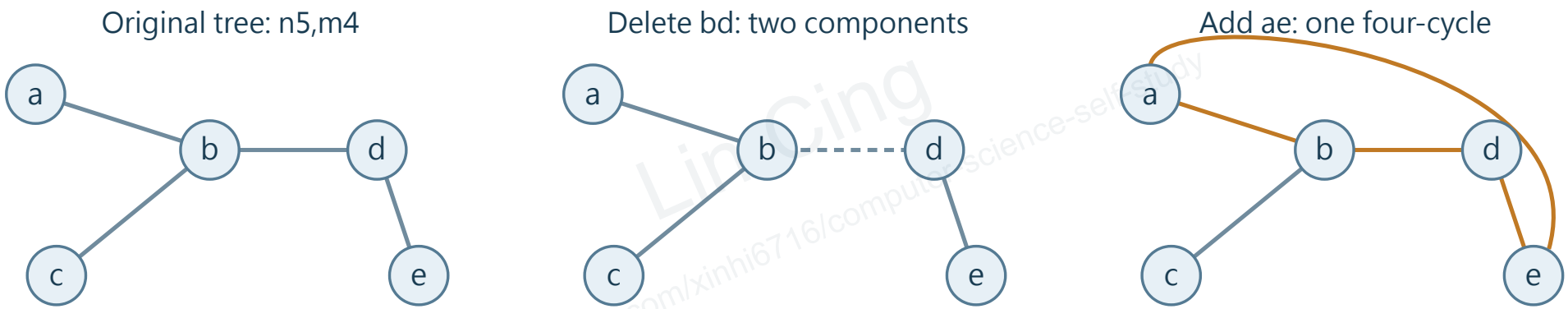
These implications establish all six equivalences. The equality $m = n - 1$ alone is insufficient. A triangle plus an isolated vertex has $n = 4, m = 3$ but is not a tree. Acyclicity alone also does not ensure connectivity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The same tree under deletion and cycle creation

Let $V = \{a, b, c, d, e\}$ and $E = \{ab, bc, bd, de\}$, so $n = 5, m = 4$. The unique simple path from $a$ to $e$ is $a, b, d, e$, of length $3$. Deleting $bd$ gives components $\{a, b, c\}$ and $\{d, e\}$. Adding $ae$ creates the four-cycle $a, b, d, e, a$.

Same vertices: remove an edge to disconnect, add one to cycle




Middle dashed edge is deleted. Right orange cycle follows a–b–d–e–a via new edge ea.

A graph with a cycle can still have a spanning tree, although the graph itself is not a tree. Minimum-spanning-tree optimization adds a weight objective that is absent from the definition of a tree.

## Rooted and ordered trees

Choose a root $r$ in a free tree. Every nonroot vertex $x$ has as its parent the preceding vertex on its unique simple path from $r$. Its other, downward neighbors are its children. Here ancestors and descendants include the node itself; proper ancestors and descendants exclude it. The subtree rooted at $x$ contains all descendants of $x$.

Depth counts edges from the root to a node. A node's height counts edges to its deepest leaf. Leaves have height $0$, the root has depth $0$, and tree height is the maximum depth. Rooted degree counts children. Graph degree also counts the parent, so it is larger by $1$ for each nonroot node.

Root the preceding example at $b$. Its children are $a, c, d$, and $d$ has child $e$. Vertices $a, c, d$ have depth $1$, and $e$ has depth $2$. The heights are $2$ for $b$, $1$ for $d$, and $0$ for the others. Vertex $b$ has both graph degree and child count $3$. Vertex $d$ has graph degree $2$ and one child.

An ordered tree also orders each child list. Replacing the order $(a, c, d)$ at $b$ by $(c, a, d)$ keeps the rooted relationships but changes the ordered tree.

Lin Cing
http://github.com/xinhi6716/computer-science-self-study

## Binary and positional trees retain empty slots

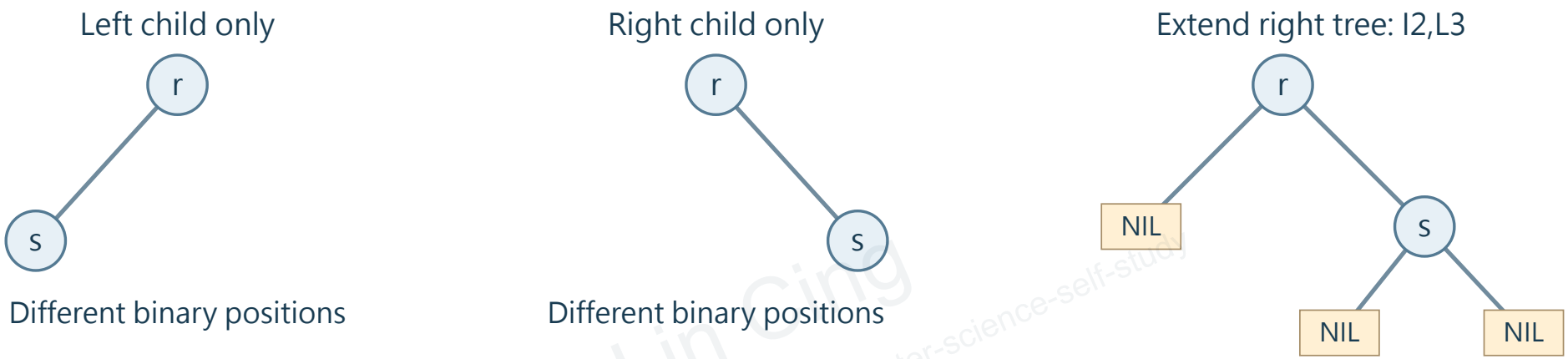
A binary tree is recursively either the empty tree `NIL` or a root with disjoint left and right binary subtrees. A single child in the left slot differs from a single child in the right slot. An ordered tree with one child has no such slot distinction, so an ordered tree of degree at most $2$ does not fully specify a binary tree.

A positional $k$-ary tree has fixed slots $1, \ldots, k$, including meaningful empty positions. Replacing every missing binary child by an external `NIL` leaf gives a full binary tree. Each internal node has two children, and the left and right positions are preserved.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Binary and positional trees retain empty slots (continued)

Empty slots retain position: left and right differ




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

For a nonempty binary tree, $n$ real nodes have $2n$ child slots. The $n-1$ real parent-child edges occupy that many slots, leaving

$$2n-(n-1)=n+1$$

missing positions. The extended tree has $n$ internal nodes and $n+1$ external leaves. Sharing one `NIL` sentinel object does not change the number of logical positions or require $n+1$ actual allocations.

## Full, complete, and height counts

A full $k$-ary tree gives each internal node exactly $k$ children. Leaves may have different depths. With $I$ internal nodes and $L$ leaves, counting edges gives

$$kI = I + L - 1, \qquad L = (k-1)I + 1.$$

For binary trees, $L = I + 1$.

CLRS's appendix uses "complete $k$-ary" to require that all leaves also have depth $h$; this is often called "perfect" elsewhere. For $k \geq 2$, level $d$ has $k^d$ nodes, and

$$L = k^h, \qquad I = \sum_{d=0}^{h-1} k^d = \frac{k^h - 1}{k - 1}.$$

At $k = 3, h = 2$, the level sizes are $1, 3, 9$, so $I = 4, L = 9$, the total is $13$, and there are $12 = 3I$ edges. For $k = 1$, a chain has $I = h, L = 1$; do not divide by $k - 1 = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Full, complete, and height counts (continued)

Heap chapters often use “complete binary” for a possibly partial final level filled from the left. Check which convention applies. A general nonempty binary tree of height $h$ has at most $2^{h+1} - 1$ nodes, giving

$$h \geq \lceil \log_2(n+1) \rceil - 1.$$

A chain has height $n - 1$. Without balance, binary-tree height need not be $\Theta(\log n)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Sources

*Bibliography*

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix B.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study